

Detailed Balance Normalizing Flow with Adaptive Prior

Qi Feng¹, Rongjie Lai², and Xuda Ye²

¹Department of Mathematics, Florida State University, Tallahassee, FL 32306

²Department of Mathematics, Purdue University, West Lafayette, IN 47907

April 23, 2026

1 Introduction

Sampling complex multi-modal distributions in high dimensions is a fundamental challenge in computational physics. The development of machine learning methods has enabled flow-based sampling as a strong alternative to classical Monte Carlo methods. At the inference stage, trained flows are capable of directly pushing particles from a simple base distribution to the target distribution, yielding samples that enjoy natural statistical independence and avoid discretization error.

Normalizing flows trained with the KL divergence have been the dominant choice in flow-based sampling. The reverse KL is used for energy-based sampling, while the forward KL is used for generative sampling. However, the KL divergence is fragile in many cases, for example, missing some of the fine structure in complex distributions. Moreover, the trained flow is essentially a diffeomorphism; connecting two distributions with vastly different shapes will lead to severe deformation.

In this paper, we mainly consider two approaches: replacing the standard KL divergence with the detailed balance divergence, and combining it with an adaptive tractable prior (essentially a Gaussian mixture) for improved sampling efficiency and accuracy.

2 Detailed balance divergence

2.1 Definition

We introduce the detailed balance loss for characterizing the discrepancy between two distributions. Let μ and ν be two probability distributions on $\mathbb{R}^d$. We define the detailed balance (DB) divergence as

$$\text{DB}(\mu\|\nu) = \int_{\mathbb{R}^d} \mu(x) \left| \nabla \left(\frac{\nu(x)}{\mu(x)} \right) \right| dx, \quad (1)$$

which is directly reminiscent of the KL divergence

$$\text{KL}(\mu\|\nu) = - \int_{\mathbb{R}^d} \mu(x) \log \frac{\nu(x)}{\mu(x)} dx.$$

Assuming that μ is the given distribution and ν is the pushforward distribution fitted to μ , the DB divergence has several notable features:

- The DB divergence is non-symmetric. Since the gradients of both $\mu(x)$ and $\nu(x)$ are required, it can only be used for energy-based sampling.
- The DB divergence is invariant under rescaling of μ . This provides a theoretical guarantee for a wide class of distributions μ , where only the unnormalized potential function is required to compute the DB divergence.

- The DB divergence is non-negative by definition due to the modulus operation. This means that even if the samples from $\mu(x)$ are not accurate, as long as we have full access to $\nabla\mu(x)$, the loss function remains accurate in comparing μ and ν .

The DB divergence (1) can be equivalently written as

$$\text{DB}(\mu\|\nu) = \int_{\mathbb{R}^d} \nu(x) |\nabla \log \mu(x) - \nabla \log \nu(x)| dx. \quad (2)$$

If we assume $\mu(x)$ takes the unnormalized potential form $e^{-U(x)}$, where $U(x)$ is the potential function, then

$$\text{DB}(\mu\|\nu) = \int_{\mathbb{R}^d} |\nabla \nu(x) + \nu(x) \nabla U(x)| dx.$$

Therefore, minimizing the DB divergence amounts to solving the PDE

$$\nabla \nu(x) + \nu(x) \nabla U(x) = 0 \quad (3)$$

in the spirit of PINNs. In fact, the PDE (3) exactly characterizes the detailed balance condition for sampling the distribution $e^{-U(x)}$ via Langevin dynamics, and thus serves as a natural metric with clear physical meaning.

2.2 Relations to other distribution metrics

The DB divergence is closely related to two classical distribution metrics: the Fisher divergence and the total variation distance. We show that it sits naturally between the two in terms of strength.

Fisher divergence. Recall that the Fisher divergence between μ and ν is defined as

$$\text{Fisher}(\nu\|\mu) = \int_{\mathbb{R}^d} \nu(x) |\nabla \log \mu(x) - \nabla \log \nu(x)|^2 dx.$$

Comparing with the equivalent form (2) of the DB divergence,

$$\text{DB}(\mu\|\nu) = \int_{\mathbb{R}^d} \nu(x) |\nabla \log \mu(x) - \nabla \log \nu(x)| dx,$$

we see that the DB divergence is precisely the L^1 counterpart of the Fisher divergence, replacing the squared L^2 norm of the score difference with the L^1 norm. By the Cauchy–Schwarz inequality and the fact that $\int \nu(x) dx = 1$, we obtain

$$\text{DB}(\mu\|\nu)^2 \leq \text{Fisher}(\nu\|\mu), \quad (4)$$

or equivalently, $\text{DB}(\mu\|\nu) \leq \sqrt{\text{Fisher}(\nu\|\mu)}$.

Total variation distance. The total variation distance between μ and ν is defined as

$$\text{TV}(\mu, \nu) = \frac{1}{2} \int_{\mathbb{R}^d} |\mu(x) - \nu(x)| dx.$$

Let $r(x) = \nu(x)/\mu(x)$ denote the density ratio and set $f(x) = r(x) - 1$. Since both μ and ν are normalized, $\int \mu(x) f(x) dx = 0$, and we can write

$$\text{TV}(\mu, \nu) = \frac{1}{2} \int_{\mathbb{R}^d} \mu(x) |f(x)| dx, \quad \text{DB}(\mu\|\nu) = \int_{\mathbb{R}^d} \mu(x) |\nabla f(x)| dx.$$

If μ satisfies a Cheeger inequality (i.e., an L^1 Poincaré inequality) with isoperimetric constant $c_P > 0$, namely

$$\int_{\mathbb{R}^d} \mu(x) |g(x) - \bar{g}| dx \leq \frac{1}{c_P} \int_{\mathbb{R}^d} \mu(x) |\nabla g(x)| dx$$

for all sufficiently regular g with $\bar{g} = \int \mu g dx$, then applying this to f with $\bar{f} = 0$ yields

$$\text{TV}(\mu, \nu) \leq \frac{1}{2c_P} \text{DB}(\mu|\nu). \quad (5)$$

In particular, log-concave distributions and Gaussian mixtures with sufficient overlap are known to satisfy such a Cheeger inequality, so for these classes the DB divergence controls the total variation distance.

Combining (4) and (5), we obtain the hierarchy

$$\text{TV}(\mu, \nu) \leq \frac{1}{2c_P} \text{DB}(\mu|\nu) \leq \frac{1}{2c_P} \sqrt{\text{Fisher}(\nu|\mu)}, \quad (6)$$

which places the DB divergence between total variation and Fisher divergence. It is weaker than the Fisher divergence yet still strong enough to control total variation under mild regularity assumptions on μ .

3 Detailed balance normalizing flow

Given a potential function $U(x)$ on $\mathbb{R}^d$, we introduce a normalizing flow approach to sample from the target distribution $\mu(x) \propto e^{-U(x)}$ based on the DB divergence (1). This involves the construction of a loss function and an adaptive choice of the prior distribution $\mu_0(x)$.

3.1 Construction of loss function

For simplicity, we assume the prior distribution $\mu_0(x) \propto e^{-U_0(x)}$ is fixed, where $U_0(x)$ is a known potential function on $\mathbb{R}^d$. Our goal is to find a flow map $F: \mathbb{R}^d \rightarrow \mathbb{R}^d$ such that $F_{\#}\mu$ matches the prior μ_0 . The pushforward is applied to the target μ rather than μ_0 intentionally, because it is typically simpler to match a flat and simple distribution than a complex multi-modal one.

The loss function is chosen as the DB divergence $\text{DB}(\mu_0 \| F_{\#}\mu)$. The probability density of the pushforward distribution is given by

$$(F_{\#}\mu)(y) = \frac{\mu(x)}{|\det J_F(x)|}, \quad y = F(x),$$

where $J_F(x) \in \mathbb{R}^{d \times d}$ is the Jacobian matrix of F at x . Note that $(F_{\#}\mu)(y)$ depends on the variable $y = F(x)$ implicitly. For convenience, we define the log-likelihood ratio $\mathcal{R}_F$ as

$$\mathcal{R}_F(x) = \log \frac{F_{\#}\mu(x)}{\mu_0(x)} + \text{const} = U_0(F(x)) - U(x) - \log |\det J_F(x)|. \quad (7)$$

The DB divergence can then be computed as

$$\text{DB}(\mu_0 \| F_{\#}\mu) = \int_{\mathbb{R}^d} \mu_0(y) \left| \frac{(F_{\#}\mu)(y)}{\mu_0(y)} \right| dy = \int_{\mathbb{R}^d} e^{-U_0(y)} |\nabla_y (e^{\mathcal{R}_F(x)})| dy = \int_{\mathbb{R}^d} e^{-U_0(x) + \mathcal{L}_F(x)} |\nabla_y \mathcal{R}_F(x)| dy.$$

Applying the change-of-variable formula with $y = F(x)$, we obtain

$$\text{DB}(\mu_0 \| F_{\#}\mu) = \int_{\mathbb{R}^d} \frac{\mu(x)}{|\det J_F(x)|} |\nabla_y \mathcal{R}_F(x)| dy = \int_{\mathbb{R}^d} \mu(x) |\nabla_y \mathcal{R}_F(x)| dx. \quad (8)$$

Therefore, computing the DB loss (8) requires evaluating the gradient of the log-likelihood ratio $\mathcal{R}_F(x)$ defined in (7) with respect to the implicit variable y .

Given the expression of $\mathcal{R}_F(x)$ in (7), a direct application of the chain rule yields

$$\begin{aligned} \nabla_y \mathcal{R}_F(x) &= \nabla U_0(F(x)) - \nabla_y (U(x) + \log |\det J_F(x)|) \\ &= \nabla U_0(F(x)) - J_F(x)^{-\top} \left(\nabla U(x) + \nabla (\log |\det J_F(x)|) \right). \end{aligned} \quad (9)$$

Substituting (9) into (8), the loss function in terms of the flow map F takes the form

$$\text{DB}(\mu_0 \| F_{\#}\mu) = \int_{\mathbb{R}^d} \mu(x) \left| \nabla U_0(F(x)) - J_F(x)^{-\top} \left(\nabla U(x) + \nabla (\log |\det J_F(x)|) \right) \right| dx. \quad (10)$$

The loss function (10) is a highly nonlinear functional of F , primarily due to the explicit presence of the Jacobian matrix $J_F(x)$ (rather than merely its determinant). Standard normalizing flow architectures such as RealNVP and NSF only support efficient computation of the determinant; computing the full Jacobian requires at least $O(d^2)$ cost. In high dimensions, this becomes a significant computational burden, even with automatic differentiation over the parameters of F .

To address this, we propose a finite-difference approach to approximate the loss function while keeping the computational cost of evaluating $\nabla_y \mathcal{R}_F(x)$ at $O(d)$. We choose a small step size $\varepsilon > 0$ and a random direction $v \in \mathbb{S}^{d-1}$, and consider the two perturbed points $F(x) \pm \varepsilon v$ near $F(x)$. By central differencing, we obtain the approximations

$$v^\top \nabla U_0(F(x)) \approx \frac{U_0(F(x) + \varepsilon v) - U_0(F(x) - \varepsilon v)}{2\varepsilon} \quad (11)$$

and

$$J_F(x)^{-1}v \approx \frac{F^{-1}(F(x) + \varepsilon v) - F^{-1}(F(x) - \varepsilon v)}{2\varepsilon}. \quad (12)$$

Substituting (11) and (12) into the expression for $\nabla_y \mathcal{R}_F(x)$ in (9), we obtain the directional derivative approximation

$$\begin{aligned} v^\top \nabla_y \mathcal{R}_F(x) &\approx \frac{U_0(F(x) + \varepsilon v) - U_0(F(x) - \varepsilon v)}{2\varepsilon} \\ &\quad - \left(\frac{F^{-1}(F(x) + \varepsilon v) - F^{-1}(F(x) - \varepsilon v)}{2\varepsilon} \right)^\top \left(\nabla U(x) + \nabla(\log |\det J_F(x)|) \right). \end{aligned} \quad (13)$$

With this approximation, the loss (8) can be estimated by repeatedly sampling x from the target distribution $\mu(x) \propto e^{-U(x)}$ and v uniformly from the unit sphere $\mathbb{S}^{d-1}$.

In particular, the loss (8) does not depend on the normalizing constant of μ_0 , and thus allows a flexible choice of the prior distribution.

3.2 Adaptive choice of prior distribution

A key practical challenge in evaluating the loss (8) is that it requires sampling from the target distribution $\mu(x) \propto e^{-U(x)}$, which is generally unavailable. However, as noted earlier, the DB divergence is non-negative by construction and remains valid even when μ is sampled with bias. This motivates the following importance sampling strategy:

1. Initialize N_0 particles $x_0^1, \dots, x_0^{N_0}$ from a broad base distribution and evolve them independently via Langevin dynamics

$$dX_t = -\nabla U(X_t)dt + \sqrt{2}dB_t$$

where B_t is the standard Brownian motion in $\mathbb{R}^d$. After sufficient burn-in, the particles $x_0^1, \dots, x_0^{N_0}$ are expected to capture the local modes of μ .

2. Choose an appropriate width parameter $\bar{\sigma}_0 > 0$ and define the Gaussian mixture

$$\bar{\mu}_0(x) = \frac{1}{N_0} \sum_{i=1}^{N_0} \mathcal{N}(x; x_0^i, \bar{\sigma}_0^2),$$

so that $\bar{\mu}_0(x)$ covers nearly all the probability mass of $\mu(x)$.

3. Rewrite the loss (8) via importance weighting as

$$\text{DB}(\mu_0 \| F_{\#}\mu) = \int_{\mathbb{R}^d} \bar{\mu}_0(x) \cdot \frac{\mu(x)}{\bar{\mu}_0(x)} \cdot |\nabla_y \mathcal{R}_F(x)| dx = \mathbb{E}_{x \sim \bar{\mu}_0} \left[\frac{\mu(x)}{\bar{\mu}_0(x)} |\nabla_y \mathcal{R}_F(x)| \right].$$

In this way, the DB loss can be computed by sampling from a fixed, tractable Gaussian mixture $\bar{\mu}_0(x)$ rather than the intractable target $\mu(x)$. Moreover, even if the importance weights $\mu(x)/\bar{\mu}_0(x)$ are computed inaccurately, the DB loss remains non-negative by construction, so the flow map F is still trained towards the correct minimizer.

We emphasize that the fixed importance sampling distribution $\bar{\mu}_0(x)$ is distinct from the flow prior $\mu_0(x)$. The former is used solely for tractable evaluation of the loss, while the latter can be adapted during training to better match the target $\mu(x)$. Specifically, we parameterize $\mu_0(x)$ as

$$\mu_0(x) \propto \sum_{i=1}^{N_0} w_i \mathcal{N}(x; x_0^i, \sigma_i^2), \quad (14)$$

or equivalently, the prior potential function is given by

$$U_0(x) = -\log \mu_0(x) = -\log \left(\sum_{i=1}^{N_0} w_i \mathcal{N}(x; x_0^i, \sigma_i^2) \right). \quad (15)$$

Here, the positions $\{x_0^i\}_{i=1}^{N_0}$ are kept fixed, while the weights $\{w_i\}_{i=1}^{N_0}$ and widths $\{\sigma_i\}_{i=1}^{N_0}$ are treated as trainable parameters. Crucially, the weights w_i do not need to be normalized, since the DB divergence is invariant under rescaling of the prior (cf. Section 2.1). This allows us to optimize the weight parameters freely. The final loss function is then

$$\text{Loss}(F, \{w_i\}_{i=1}^{N_0}, \{\sigma_i\}_{i=1}^{N_0}) = \mathbb{E}_{x \sim \bar{\mu}_0} \left[\frac{\mu(x)}{\bar{\mu}_0(x)} |\nabla_y \mathcal{R}_F(x)| \right] + \text{PriorPotential}(\{w_i\}_{i=1}^{N_0}, \{\sigma_i\}_{i=1}^{N_0}), \quad (16)$$

where the prior potential on w_i and σ_i can be conveniently specified via a Gamma distribution. Note that if F were perfectly trained to minimize the DB loss, the total loss (16) would be minimized exactly when all w_i and σ_i coincide at their prior mean, ensuring the theoretical stability of the optimization.

Remark *Since the Gaussian distribution has unbounded support, in practice one may replace it with an isotropic cosine cutoff distribution of finite width, which is more compatible with NSF architectures that operate on a bounded domain.*

3.3 One-step importance sampling at inference

After training with the total loss (16), we obtain the trained flow map F along with the optimized parameters $\{w_i\}_{i=1}^{N_0}$ and $\{\sigma_i\}_{i=1}^{N_0}$, which define a tractable Gaussian mixture prior μ_0 . Since the log-likelihood ratio $\mathcal{R}_F(x)$ is available in closed form via (7), we can sample from the target distribution $\mu(x)$ using the following procedure:

1. Draw N samples $y_1, \dots, y_N$ from the prior distribution μ_0 .
2. Compute the inverse flow positions $x_i = F^{-1}(y_i)$ for $i = 1, \dots, N$.
3. Evaluate the log-likelihood ratio $\mathcal{R}_F(x_i) = U_0(y_i) - U(x_i) - \log |\det J_F(x_i)|$ for $i = 1, \dots, N$.
4. Compute the importance weights $w_i = \exp(\mathcal{R}_F(x_i))$ for $i = 1, \dots, N$ and normalize so that $\sum_{i=1}^N w_i = 1$.
5. The resulting weighted empirical distribution

$$\hat{\mu}(x) = \sum_{i=1}^N w_i \delta(x - x_i)$$

provides an approximation to the target distribution $\mu(x)$. If samples with uniform weights are desired, one may resample from $\hat{\mu}$.

This procedure requires only a single flow evaluation per sample. Its effectiveness relies on the assumption that the adapted prior μ_0 already captures the dominant structure of the target distribution μ , so that the importance weights remain well-behaved.

4 Implementation

The implementation uses **PyTorch** as the primary framework and the **zuko** package for the neural spline flow (NSF) architecture. The Detailed Balance Normalizing Flow (DBNF) method is organized as a standalone Python package **dbnf** with the following source files:

- **potential.py**. Defines a base **Potential** class and specific potential subclasses (e.g., double-well, Gaussian mixture). Each subclass stores the potential name and parameters, and supports batched evaluation of $U(x)$. The gradient $\nabla U(x)$ is computed via **torch** automatic differentiation.
- **distribution.py**. Implements tractable distribution classes such as Gaussian mixtures and cosine cutoff distributions. Log-densities are evaluated using the log-sum-exp trick for numerical stability. Also provides the prior potential regularizer for the adaptive parameters $\{w_i\}$ and $\{\sigma_i\}$, as well as utilities for resampling and computing the effective sample size (ESS).
- **flow.py**. Wraps **zuko**’s NSF implementation, exposing the forward map F , its inverse F^{-1} , and the log-determinant $\log |\det J_F(x)|$.
- **dbloss.py**. Evaluates the DB loss $\text{DB}(\mu_0 \| F_{\#}\mu)$ given the prior μ_0 , the target potential U , and the flow map F , using the finite-difference approximation (13).

Outside the package, the full pipeline consists of the following scripts:

1. **global_parameters.py**. Specifies all global configuration, including: choice of target potential and dimension d ; number of particles N_0 and burn-in steps for the Langevin mode-finding stage; initial width $\bar{\sigma}_0$ for the importance sampling distribution $\bar{\mu}_0$; finite-difference step size ε and number of random directions per sample for evaluating (13); NSF architecture settings (number of layers, hidden dimensions, spline bins); learning rates and number of epochs for both the flow optimizer and the prior optimizer; and output paths.
2. **set_prior.py**. Constructs the importance sampling distribution $\bar{\mu}_0(x)$ by running short Langevin trajectories, and initializes the adaptive prior parameters $\{w_i\}$ and $\{\sigma_i\}$.
3. **train_flow.py**. Combines the DB loss and the prior potential regularizer into the total loss (16). Training proceeds by alternating two optimizers: one updates the flow parameters of F , and the other updates the adaptive prior parameters $\{w_i\}$ and $\{\sigma_i\}$. This alternating scheme decouples the flow and prior updates, allowing separate learning rates and improving training stability.
4. **generate_samples.py**. Performs one-step importance sampling (Section 3.3), generates target samples, and reports the ESS.
5. **plot_results.py**. Visualizes the generated samples and diagnostic statistics.

5 Detailed Balance Score Matching

Although DBNF cannot be directly used for generative sampling, it can be combined with standard approaches such as score matching. Suppose that samples from a multi-modal target distribution $\mu(x)$ are available, and we seek a fast sampling method. As before, we choose μ_0 as a tractable adaptive prior and train a flow map F to minimize the DB loss $\text{DB}(\mu_0 \| F_{\#}\mu)$. Once μ_0 and F are trained, the distributions μ_0 and $F_{\#}\mu$ are close, and the remaining gap can be bridged by a standard generative layer such as score-based diffusion, which is able to overcome the geometric constraints of diffeomorphisms.

The central challenge remains the evaluation of the DB loss (10). In this setting, samples from $\mu(x)$ are available, so importance sampling is not needed. The main difficulty is that the gradient $\nabla U(x)$ is not directly accessible from samples alone.

Nevertheless, we can employ implicit score matching to obtain an approximation to $\nabla U(x)$. Let $s(x) \in \mathbb{R}^d$ denote a parametric score model. The implicit score matching loss is

$$\int_{\mathbb{R}^d} \mu(x) |s(x) - \nabla U(x)|^2 dx = \int_{\mathbb{R}^d} \mu(x) (|s(x)|^2 + 2\nabla \cdot s(x)) dx, \quad (17)$$

where the right-hand side depends only on samples from $\mu(x)$ and does not require explicit knowledge of $U(x)$. Minimizing (17) yields $s(x) \approx \nabla U(x)$, which can then be substituted into the DB loss (10). With this surrogate gradient, we train the flow map F and the adaptive prior $\mu_0(x)$ as in DBNF. Finally, a score-based diffusion layer is added between μ_0 and $F_{\#}\mu$ to close the remaining gap. Since μ_0 and $F_{\#}\mu$ are already close after DBNF training, the diffusion process only needs to run for a short time, which is expected to yield efficient and high-quality generative sampling.